

Algorithm Reference

What each module does, step by step, and why

Authors: Md Mostakim (ID: 2305063)

Samiul Basir Adry (ID: 2305071)

Course: CSE 220 — Digital Signal Processing

This document walks through the DSP theory and the exact algorithmic steps behind every module of DSP Studio, in plain English, alongside the governing formulas and real screenshots from the application.

Contents

How to Read This Document	2
1 Amplitude & Gain	2
2 Time-Domain Processing	3
2.1 Convolution	3
2.2 Echo	3
2.3 Delay	4
3 Spectral Detection — Pitch	5
4 Sampling & Quantization	6
4.1 Quantization	6
4.2 Sample-and-Hold (Downsampling)	7
5 Frequency & Filtering	7
6 Voice Morphing — Phase Vocoder	8
6.1 Pitch Shift & Time Stretch — the Phase Vocoder	9
6.2 Robot (Zero Phase) and Whisper (Random Phase)	9
7 Spectral Portal — Spectrogram Masking	10
8 Studio — Multi-Operation Signal Chain	11
9 Technology Summary	12

How to Read This Document

DSP Studio is a web application with 7 interactive modules plus a Studio page that chains them together. Every module lets you upload or record an audio clip, and a Python/FastAPI backend (NumPy, SciPy, Librosa) runs the real signal-processing algorithm on it — nothing is faked or pre-rendered. This document explains, module by module, what each algorithm actually does: first in plain English as a numbered list of steps, then with the formula that step corresponds to, and finally with a real screenshot of the result.

1 Amplitude & Gain

What it does

Amplitude scaling (gain) is the simplest possible DSP operation: it changes how loud a signal is without changing its shape, timing, or frequency content at all.

Algorithm

1. Read the input signal $x[n]$ sample by sample.
2. Multiply every sample by a single constant number A , the gain factor. If $A > 1$ the signal gets louder; if $A < 1$ it gets quieter; if A is negative the waveform is also flipped upside down (phase-inverted).
3. Write the scaled value out as $y[n]$.

$$y[n] = A \cdot x[n]$$

Because every sample is scaled by the exact same number, the output waveform is a perfect scaled copy of the input — same zero-crossings, same shape, just taller or shorter peaks. The app also reports peak amplitude, RMS (average loudness), and clipping percentage before and after, so a gain that pushes samples past ± 1 (which would distort on playback) is visible immediately.

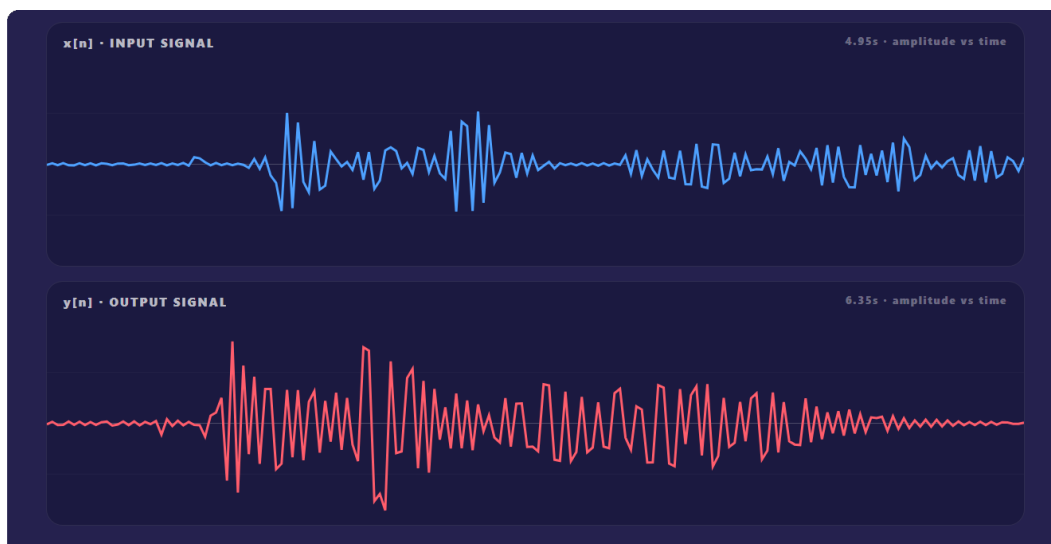


Figure 1: Input $x[n]$ (top) vs. output $y[n]$ (bottom) for a gain of $A = 2$.

2 Time-Domain Processing

This module covers three related effects — Convolution, Echo, and Delay — that all operate directly on the raw sample stream (“time domain”, as opposed to the frequency domain used later).

2.1 Convolution

What it does

Convolution is the mathematical operation that describes how a signal is reshaped by passing through any linear system — for example, a room’s acoustics, which turns a sharp handclap into a long, blurred decay (its *impulse response*). Convoluting a signal with a recorded impulse response $h[n]$ makes the signal sound as if it was played inside that room.

Algorithm

1. Take the input signal $x[n]$ and an impulse response $h[n]$.
2. **Reverse** h in time, i.e. flip it back-to-front.
3. **Slide** the reversed h across x , one sample position at a time.
4. At every position, **multiply** the overlapping samples of x and the flipped/shifted h pointwise, then **sum** all those products — that single sum becomes one output sample $y[n]$.
5. Repeat for every shift, from the moment the two signals first touch until the reversed h has slid completely past x .

$$y[n] = \sum_k x[k] h[n - k]$$

Implementation note

Computing this sum directly for every single output sample is $O(N^2)$ and far too slow for a multi-second audio clip. The backend instead computes it via *FFT convolution* (`scipy.signal.fftconvolve`): transform both signals to the frequency domain, multiply their spectra pointwise, then transform back. This is mathematically the exact same result, just computed in $O(N \log N)$ time. The on-screen animation, however, plays out the literal reverse-slide-multiply-accumulate definition above (on a reduced number of points, so it stays watchable) so the algorithm itself is visible, not just its output.

2.2 Echo

What it does

Echo layers several decaying, evenly-spaced repeats of the signal on top of itself — like shouting in a canyon and hearing your voice come back multiple times, each time a little quieter.

Algorithm

1. Start with the original (“dry”) signal as the base of the output.
2. Make a copy of the signal delayed later in time by D samples.
3. Scale that delayed copy down by a decay factor α (where $0 < \alpha < 1$) and add it into the output.
4. **Feed that result back** and repeat steps 2–3 a total of R times: each new repeat is delayed by another D samples and decayed by another factor of α , so the r -th repeat is scaled by α^r .
5. The final output is the sum of the dry signal and all R decaying, delayed copies.

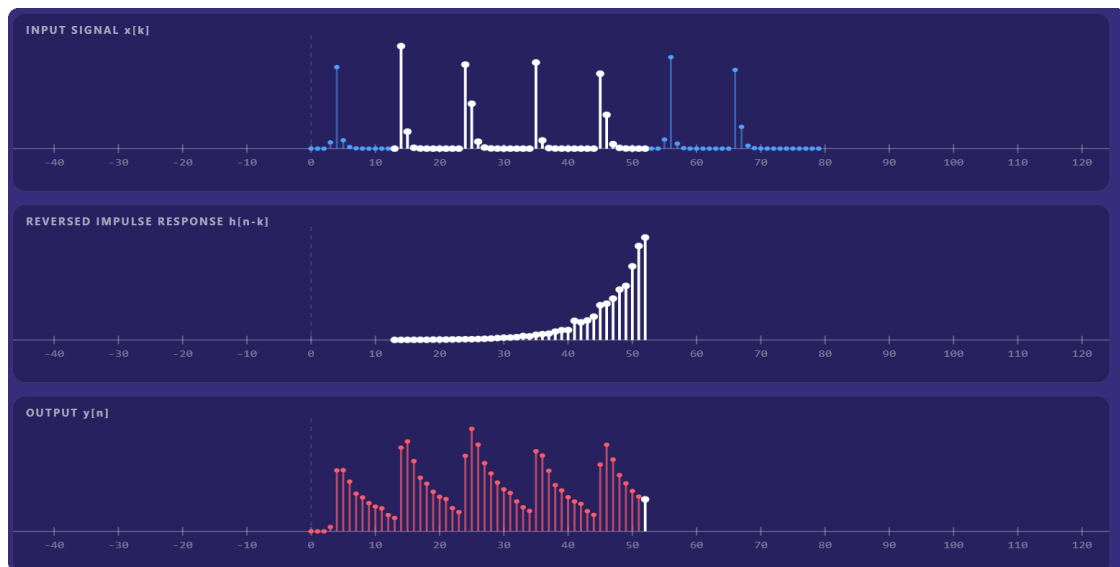


Figure 2: The input $x[k]$, the reversed impulse response $h[n-k]$ sliding across it, and the accumulating output $y[n]$.

$$y[n] = x[n] + \sum_{r=1}^R \alpha^r x[n - rD]$$



Figure 3: Input $x[n]$ vs. output $y[n]$ — notice the repeated, progressively quieter reflections after each original spike.

2.3 Delay

What it does

Delay is the simpler cousin of Echo: instead of a chain of repeats, it adds back exactly *one* clean, delayed copy of the signal — no feedback loop.

Algorithm

1. Start with the original (dry) signal.
2. Make **one** copy of the signal, delayed by D samples. This copy is never fed back into itself the way Echo's is.
3. Scale that single copy by a wet-mix factor α .
4. Add it to the original signal.

$$y[n] = x[n] + \alpha \cdot x[n - D]$$

Both effects use the exact same building block — a scaled, time-shifted copy $\alpha x[n - D]$ — Echo simply applies it recursively R times, while Delay applies it exactly once.



Figure 4: Input $x[n]$ vs. output $y[n]$ — a single, clean, delayed repeat.

3 Spectral Detection — Pitch

What it does

This module estimates the *dominant pitch* of a signal — the strongest musical note present — by looking at its frequency content rather than its raw waveform, then synthesizes a pure tone at that frequency so the detection can be verified by ear.

Algorithm

1. Apply a Hann window to the signal (tapering its edges to zero) to reduce spectral leakage before transforming it.
2. Compute the **FFT** of the windowed signal, giving the magnitude spectrum $|X(f)|$: how much energy exists at every frequency.
3. Restrict the search to a user-chosen range $[f_{\min}, f_{\max}]$, so DC offset, sub-bass rumble, and in-audible ultrasonic content cannot accidentally win.
4. Find the frequency bin with the **largest magnitude** inside that range — this is the detected dominant frequency f_0 .
5. Convert f_0 to the nearest musical note name using the standard equal-temperament formula ($A4 = 440$ Hz).

6. **Synthesize** a short, pure sine wave at exactly f_0 , so the answer can be checked by listening to it alongside the original.

$$f_0 = \arg \max_{f \in [f_{\min}, f_{\max}]} |X(f)| \quad y[n] = A \sin(2\pi f_0 n / f_s)$$



Figure 5: Input spectrum $X(f)$ (left) — energy spread across a fundamental and its overtones — vs. the synthesized tone's spectrum $Y(f)$ (right), a single sharp spike exactly at f_0 .

4 Sampling & Quantization

This module demonstrates the two ways digitizing audio can lose information: coarser *amplitude* resolution (quantization) and coarser *time* resolution (sample-and-hold, a stand-in for a lower sample rate).

4.1 Quantization

What it does

A digital sample is stored using a fixed number of bits, which means only a finite set of amplitude values is representable. Reducing that bit depth forces every sample to snap to the nearest of a smaller set of levels, which is audible as a gritty “crunch” called quantization noise.

Algorithm

1. Choose how many bits b to keep (e.g. 4 bits = 16 levels).
2. Compute the step size Δ that spans the full amplitude range $[-1, 1]$ using 2^b evenly spaced levels.
3. For every sample, divide by Δ , **round** to the nearest whole number, then multiply back by Δ — this snaps the sample onto the nearest allowed level.
4. Clip the result to $[-1, 1]$ to guarantee it stays in range.

$$y[n] = \Delta \cdot \text{round}\left(\frac{x[n]}{\Delta}\right), \quad \Delta = \frac{2}{2^b - 1}$$

4.2 Sample-and-Hold (Downsampling)

What it does

This simulates what happens when a signal is captured at a lower sample rate: instead of measuring a fresh value at every time step, the last measured value is simply *held* until the next real measurement arrives.

Algorithm

1. Choose a hold factor N — the effective rate drops by a factor of N .
2. Walk through the signal in consecutive blocks of N samples.
3. For each block, take the value of its **first** sample and repeat (hold) that same value for every position in the block, instead of reading a new value each time. This is exactly what a zero-order-hold DAC does.

$$y[n] = x(N \lfloor n/N \rfloor)$$

The output stays the same length as the input (each held sample simply repeats N times), producing a “staircase” shape. This staircase is precisely what makes aliasing audible — the abrupt jumps introduce high-frequency energy that was never in the original signal.

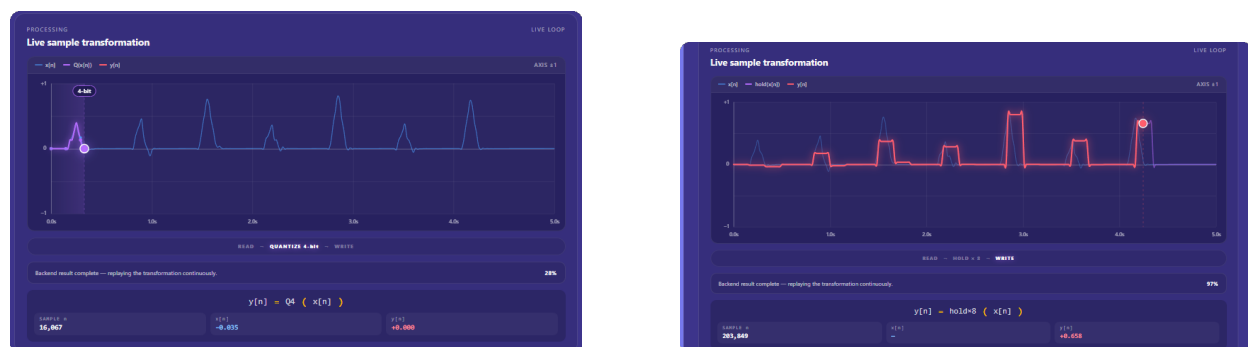


Figure 6: Live transformation panels: quantization to 4-bit (left) and sample-and-hold at $\times 8$ (right). Both show the formula applied to the current sample in real time.

5 Frequency & Filtering

What it does

A digital filter reshapes a signal’s frequency content — e.g. removing everything above a cutoff frequency (lowpass) or everything outside a band (bandpass). This module lets you design filters from five classical IIR families (Butterworth, Chebyshev I, Chebyshev II, Elliptic, Bessel) plus an “ideal” brick-wall filter, at four band types (lowpass, highpass, bandpass, bandstop).

Algorithm

1. Choose a filter family, a band type, cutoff frequency (or frequencies for band filters), and a filter order N (higher order = steeper roll-off, closer to an ideal brick wall).
2. **Design the filter** using that family’s standard design equations, producing the filter as **SOS** — *second-order sections*.
3. **Apply the filter** to the signal using `sosfiltfilt`: run the SOS cascade across the signal once forwards, then once more backwards (on the time-reversed result).

$$Y(f) = X(f) \cdot H(f)$$

What is SOS, and why filter twice?

A digital IIR filter is normally described by a transfer function — one set of numerator/denominator coefficients. For a high order (say, 8th order) or a very narrow band, those coefficients can span an enormous numeric range, and ordinary floating-point rounding error in that form is severe enough to make the filter numerically unstable — in the worst case, the output becomes entirely NaN.

Second-order sections (SOS) solve this by never building that one giant filter at all. Instead, the design is expressed as a *cascade* (a chain) of many small, simple 2-pole/2-zero (“biquad”) filter stages, each one individually well-conditioned, multiplied together to form the overall response. Filtering the signal through the small stages one after another gives the identical mathematical result as the single high-order filter would, but stays numerically stable at any order.

Running the SOS cascade only *forwards* would also shift the signal in time (phase distortion) — later frequencies would lag behind earlier ones. `sosfiltfilt` fixes this by filtering the signal once forwards and once more backwards (i.e. filtering the time-reversed signal, then reversing the result back). The delay introduced by the forward pass is exactly cancelled by the backward pass, so the final output lines up in time with the input perfectly — this is called *zero-phase filtering*. The cost is that the signal is effectively filtered twice (squaring the magnitude response), which the design step already accounts for.

The one exception is the **“ideal” family**: instead of designing any IIR filter at all, it takes the FFT of the signal, zeroes out every frequency bin outside the desired band directly, and inverse-FFTs back — a perfect brick-wall cutoff with no transition band, used here mainly as a textbook comparison against the real, imperfect filter families.

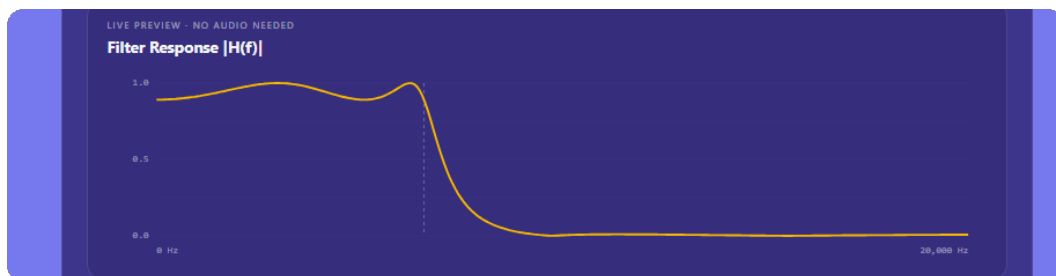


Figure 7: The filter’s own frequency response $|H(f)|$ for a lowpass Butterworth design — flat in the passband, then rolling off past the cutoff f_c .

6 Voice Morphing — Phase Vocoder

What it does

Every frequency-domain bin of a signal has two independent pieces of information: a **magnitude** $|X(f)|$ (how much energy is at that frequency) and a **phase** $\angle X(f)$ (that frequency component’s timing/alignment). This module manipulates those two pieces *independently* to produce four different effects.

$$X(f) = |X(f)| \cdot e^{j\angle X(f)}$$

6.1 Pitch Shift & Time Stretch — the Phase Vocoder

Pitch-shifting and time-stretching both rely on the same underlying engine, called a *phase vocoder*. Time-stretching changes how long the signal lasts while keeping every frequency exactly where it was; pitch-shifting reuses that same time-stretch and then resamples the result to restore the original duration, which moves the pitch.

Algorithm

1. **STFT**: split the audio into overlapping windowed frames and FFT each one, analyzing magnitude and phase of every bin *separately* for every frame.
2. **Time-mapping**: decide new frame positions for the desired stretch rate r . The output (synthesis) frames stay evenly spaced as usual, but the corresponding input (analysis) read-position advances by r times the normal spacing — so for $r < 1$ frames are read more densely (stretching the sound out), and for $r > 1$ more sparsely (compressing it). This desired read-position is usually a fractional index, landing *between* two frames that were actually computed.
3. **Magnitude interpolation**: since the desired analysis position falls between two real frames, linearly interpolate the magnitude spectra of those two neighboring frames to estimate the magnitude at the in-between position.
4. **Phase updating**: phase cannot simply be interpolated the same way — doing so produces a smeared, “phasy” sound. Instead, for each frequency bin, the algorithm measures how much the phase actually advanced between consecutive analysis frames, uses that to estimate the bin’s *true instantaneous frequency*, and then *accumulates* (integrates) the phase forward from the previous synthesis frame using that true frequency, at the new synthesis hop spacing. This keeps the phase relationships between frames coherent across the whole clip — it is the specific trick that makes a phase vocoder work, and the reason it isn’t called just a “magnitude vocoder”.
5. **Reconstruct**: combine the interpolated magnitude with the newly accumulated phase into a new complex spectrum per frame, inverse-FFT each frame, and overlap-add the frames back into one continuous time-domain signal. At this point the signal’s *duration* has changed by the stretch ratio, but every frequency is still where it started.
6. **Pitch shift only**: take that time-stretched result and **resample** it back to the original sample count. Resampling changes the playback rate, which shifts every frequency proportionally — undoing the duration change while leaving the pitch shifted. Net effect: only the pitch moves; the duration returns to exactly what it was.

6.2 Robot (Zero Phase) and Whisper (Random Phase)

What it does

These two effects skip the whole time-mapping/interpolation machinery above — they take a single STFT of the whole clip and only rewrite its phase, leaving every magnitude value completely untouched.

Algorithm

1. Take one STFT of the signal, obtaining magnitude $|X(f)|$ and phase $\angle X(f)$ for every bin of every frame.
2. **Robot**: set every bin’s phase to exactly 0, keeping magnitude unchanged. **Whisper**: replace every bin’s phase with an independent random value instead, also keeping magnitude unchanged.
3. Inverse-STFT the result back into audio.

$$\begin{aligned} \text{Robot: } Y(f) &= |X(f)| \cdot e^{j \cdot 0} = |X(f)| \\ \text{Whisper: } Y(f) &= |X(f)| \cdot e^{j\theta}, \theta \sim \text{random} \end{aligned}$$

Since *nothing* about $|X(f)|$ changes in either case, these effects are direct proof that phase — not magnitude — carries a voice’s sense of timing and “voicedness”. Zeroing phase collapses every frequency component to start at the same instant, producing a flat, buzzy monotone (Robot). Randomizing phase instead decorrelates the components without collapsing them, producing a breathy, unvoiced, whisper-like texture.

Named presets (*Vader-style*, *Protocol Droid*, *Radio Trooper*, *Cylon Monotone*) start from this same zero-phase Robot base and layer additional, already-covered effects on top of it — pitch shifting, lowpass/bandpass filtering, soft-clip distortion, ring modulation, and reverb — to build more specific character voices.



Figure 8: Robot mode: input magnitude/phase (left) vs. output magnitude/phase (right). Magnitude is unchanged; the scattered input phase has been flattened to zero.

7 Spectral Portal — Spectrogram Masking

What it does

This module lets you literally paint on a picture of the sound’s frequency content over time, erasing or isolating whichever time-frequency regions you select — for example, removing a specific tone that only happens between 1 and 3 seconds, without touching anything else.

Algorithm

- 1. Build the spectrogram:** split the signal into overlapping windowed frames and FFT each one (an STFT), giving a 2-D grid where the horizontal axis is time, the vertical axis is frequency, and each cell’s value is $|X(k, m)|$ in decibels for frequency bin k and time frame m .
- 2. Colour the grid** on a purple-to-gold scale, normalized between that clip’s own quietest and loudest cell — quiet cells render dark purple, loud cells render bright gold.
- 3. Paint KEEP or ERASE rectangles** directly on that image, in real Hz/second coordinates.
- 4. Build a mask $M(k, m)$** from those rectangles: if any “keep” region was drawn, everything starts silent ($M = 0$) and only the keep regions are set to 1 (an isolating mask); otherwise everything

starts audible ($M = 1$) and only the erase regions are set to 0 (a subtractive mask). Later regions win where two overlap, matching paint order.

5. **Feather the mask:** apply a Gaussian blur across both axes of the mask before using it. A hard rectangular cut would create sharp discontinuities in the spectrum, which the inverse transform would hear as ringing/"musical noise"; blurring softens every edge into a smooth ramp instead.
6. **Apply the mask:** multiply the mask elementwise into the complex STFT, $X'(k, m) = M(k, m) \cdot X(k, m)$.
7. **Inverse STFT** (with overlap-add) reconstructs the edited time-domain audio from what remains of the spectrum.

$$X'(k, m) = M(k, m) \cdot X(k, m)$$

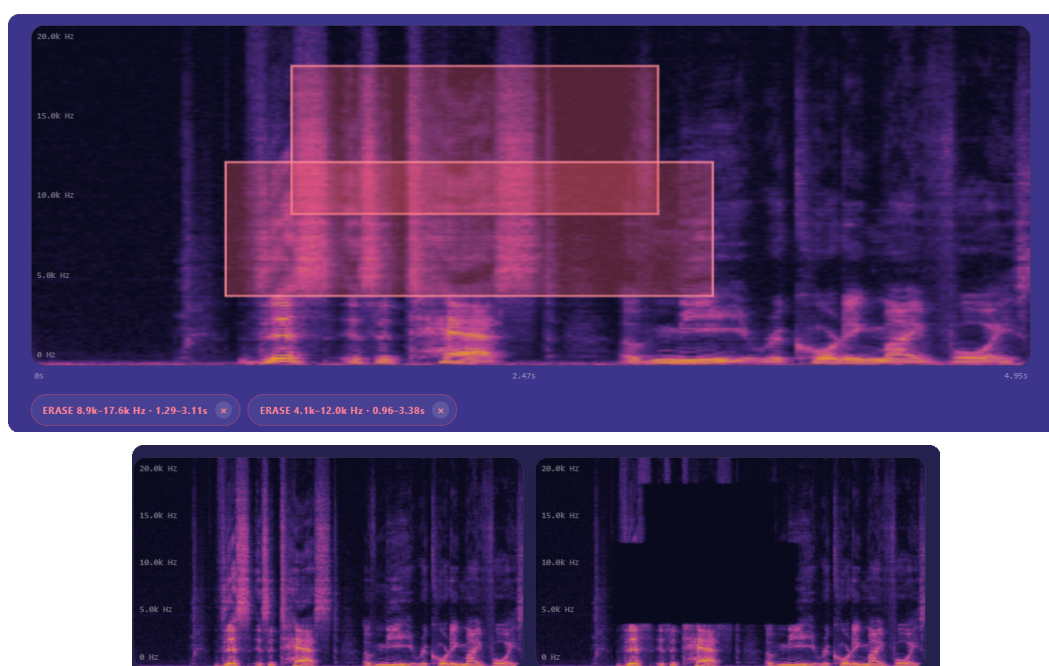


Figure 9: Top: two erase regions drawn on the spectrogram, with their Hz/time bounds. Bottom: the resulting spectrogram before (left) and after (right) the mask is applied — the erased region is now silent.

8 Studio — Multi-Operation Signal Chain

What it does

The Studio page lets any of the effects above be composed into one ordered pipeline, so complex processing chains (e.g. filter → amplify → reverb) can be built and run as a single job.

Algorithm

1. Upload or record one input signal, used as the chain's starting point.
2. Add operation steps to the chain, in any order (up to 25 steps), each with its own parameters.
3. Run the chain: the backend executes each step in sequence, and **every step's output becomes the next step's input**.

4. Each step reports its own elapsed time and signal levels, and the final result is added to a comparison history so different chains/parameters can be judged side by side.

$$y[n] = f_K(f_{K-1}(\cdots f_1(x[n]) \cdots))$$

where $f_1, \dots, f_K$ are the chain's steps in order, and each f_i is exactly one of the algorithms described in the sections above.

9 Technology Summary

- **Backend engine:** Python, FastAPI, NumPy, SciPy, Librosa, soundfile.
- **Frontend interaction:** React, Vite, Canvas-based live visualizations.
- **DSP methods covered:** FFT & spectrograms, convolution, IIR filtering with SOS/zero-phase filtering, quantization & aliasing, the phase vocoder, and frequency-domain masking.

Every module above is computed for real on the backend from the audio you provide — the figures in this document are actual screenshots of the application's own output, not illustrations.